AULA PRÁTICA • .NET + HTML + CSS + JAVASCRIPT

Mini DataSales | Código completo

Todas as linhas essenciais explicadas com exemplos que podem ser copiados e adaptados.

DO FRONTEND ESTÁTICO
AO DASHBOARD CONECTADO

ETAPA DA AULA

O que vamos construir

Uma aplicação funcional, sem banco de dados por enquanto.

01 Backend em C# com duas rotas GET

02 Frontend HTML com indicadores e tabela

03 CSS para organizar e estilizar a tela

04 JavaScript com fetch, JSON e atualização da interface

ETAPA DA AULA

Arquivos e fluxo

Coloque o frontend em wwwroot para acessar tudo pelo mesmo servidor .NET.

Program.cs → configura servidor, dados e APIs

wwwroot/index.html → estrutura da página

wwwroot/css/style.css → aparência

wwwroot/js/script.js → carrega e exibe dados

CÓDIGO EXPLICADO

Criando o projeto

No Windows e macOS, os comandos podem ser executados no terminal do VS Code.

CÓDIGO	EXPLICAÇÃO
`dotnet new web -n minidatasales`	Gera uma aplicação ASP.NET Core minimalista.
`cd minidatasales`	Entra na pasta do projeto criado.
`code .`	Abre a pasta no VS Code (caso o comando code esteja disponível).
`mkdir wwwroot`	Cria a pasta pública onde ficará o frontend.

CÓDIGO EXPLICADO

Program.cs • inicialização

CÓDIGO	EXPLICAÇÃO
`var builder = WebApplication.CreateBuilder(args);`	Prepara a aplicação e sua configuração.
`var app = builder.Build();`	Constrói o servidor Web.
`app.UseDefaultFiles();`	Permite servir index.html ao acessar a raiz /.
`app.UseStaticFiles();`	Libera acesso aos arquivos de wwwroot (HTML, CSS, JS).

CÓDIGO EXPLICADO

Program.cs • declarando dados

CÓDIGO

```
var vendas = new[]

{

new { Produto = "Notebook", Categoria = "Tecnologia",
Valor = 3500m },

new { Produto = "Mouse", Categoria = "Tecnologia", Valor
= 100m },

...

};
```

EXPLICAÇÃO

Declara um conjunto de registros guardados na memória.

Abre o bloco de registros.

Cria uma venda: Produto, Categoria e Valor; m indica decimal.

Adiciona um segundo registro.

Repita o padrão para Cadeira, Mesa e Monitor.

Encerra o conjunto de vendas.

CÓDIGO EXPLICADO

Program.cs • rotas GET

CÓDIGO

```
app.MapGet("/api/vendas", () => vendas);


app.MapGet("/api/indicadores", () =>


{


var receitaTotal = vendas.Sum(v => v.Valor);


var totalVendas = vendas.Length;
```

EXPLICAÇÃO

Quando alguém acessa /api/vendas, o backend devolve os registros em JSON.

Abre a rota que calcula os indicadores.

Inicia o bloco executado quando a rota é chamada.

Soma os valores de todas as vendas.

Conta quantos registros existem.

CÓDIGO EXPLICADO

Program.cs • cálculo e resposta

CÓDIGO	EXPLICAÇÃO
`var ticketMedio = totalVendas == 0 ? 0 : receitaTotal / totalVendas;`	Calcula a média; protege o cálculo contra divisão por zero.
`return new { receitaTotal, totalVendas, ticketMedio };`	Cria um objeto anônimo: a API o serializa em JSON.
`});`	Fecha e registra a rota de indicadores.
`app.Run();`	Inicia o servidor e mantém a aplicação atendendo requisições.

ETAPA DA AULA

Duas rotas, duas respostas

Ambas usam os mesmos dados do backend.

/api/vendas → lista com produto, categoria e valor

/api/indicadores → receitaTotal, totalVendas, ticketMedio

GET significa: consultar informações

JSON é o formato que o frontend receberá

CÓDIGO EXPLICADO

index.html • início da página

CÓDIGO

```
<!DOCTYPE html>

<html lang="pt-BR">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width,
initial-scale=1.0">

<title>Mini DataSales</title>
```

EXPLICAÇÃO

Indica que o documento usa HTML5.

Marca o início da página e seu idioma.

Abre a área de metadados e recursos.

Permite exibir corretamente os caracteres em português.

Adapta a escala da página a celulares.

Nome da aba do navegador.

CÓDIGO EXPLICADO

index.html • ligando CSS e JS

CÓDIGO	EXPLICAÇÃO
`<link rel="stylesheet" href="/css/style.css">`	Importa os estilos guardados em wwwroot/css.
`<script src="/js/script.js" defer></script>`	Carrega o JavaScript; defer espera o HTML ser analisado.
`</head>`	Encerra o cabeçalho.
`<body>`	Inicia o conteúdo visível.
`<main class="container">`	Define a área principal para organizar o layout.

CÓDIGO EXPLICADO

index.html • cabeçalho

CÓDIGO

```
<header>


<p class="tag">DASHBOARD DE VENDAS</p>


<h1>Mini DataSales</h1>


<p>Indicadores atualizados pela API em .NET</p>


</header>
```

EXPLICAÇÃO

Agrupa a apresentação inicial do dashboard.

Texto auxiliar que receberá uma cor especial pelo CSS.

Título principal.

Explica a origem dos dados.

Fecha a área de cabeçalho.

CÓDIGO EXPLICADO

index.html • cards (indicadores)

CÓDIGO	EXPLICAÇÃO
`<section class="cards" aria-label="Indicadores">`	Agrupa os três cartões; aria-label ajuda leitores de tela.
`<article class="card"><h2>Receita total</h2><p id="receita">Carregando...</p></article>`	Cria o card e fornece o ID receita que o JS atualizará.
`<article class="card"><h2>Total de vendas</h2><p id="total">Carregando...</p></article>`	ID total identifica a quantidade de vendas.
`<article class="card"><h2>Ticket médio</h2><p id="ticket">Carregando...</p></article>`	ID ticket identifica o valor médio.
`</section>`	Encerra a seção de cartões.

CÓDIGO EXPLICADO

index.html • tabela

CÓDIGO

```
<section class="painel">
```

```
<h2>Vendas registradas</h2>
```

```
<p id="status" role="status">Buscando dados...</p>
```

```
<table>
```

```
<thead><tr><th>Produto</th><th>Categoria</th><th>Valor</th></tr></th
ead>
```

```
<tbody id="tabela-vendas"></tbody>
```

EXPLICAÇÃO

Cria a área que abriga a lista de vendas.

Título da tabela.

Exibe mensagens de carregamento e erro.

Abre a tabela HTML.

Cria os títulos das três colunas.

O JavaScript acrescentará as linhas aqui.

CÓDIGO EXPLICADO

index.html • final

CÓDIGO	EXPLICAÇÃO
`</table>`	Fecha a tabela.
`</section>`	Fecha o painel de vendas.
`</main>`	Fecha o conteúdo principal.
`</body>`	Fecha o corpo da página.
`</html>`	Encerra o documento HTML.

CÓDIGO EXPLICADO

style.css • base visual

CÓDIGO	EXPLICAÇÃO
`:root { font-family: Arial, sans-serif; color: #18243a; background: #f4f7fb; }`	Define fonte e cores globais.
`* { box-sizing: border-box; }`	Inclui bordas e padding na largura calculada.
`body { margin: 0; }`	Remove a margem padrão do navegador.
`.container { max-width: 1050px; margin: 0 auto; padding: 40px 24px; }`	Centraliza e limita a largura do conteúdo.
`header { margin-bottom: 30px; }`	Cria espaço abaixo do cabeçalho.

CÓDIGO EXPLICADO

style.css • cards e tipografia

CÓDIGO	EXPLICAÇÃO
`h1 { font-size: 36px; margin: 8px 0; }`	Dá destaque ao título.
`.tag { color: #5654c9; font-weight: bold; letter-spacing: 2px; }`	Configura etiqueta do cabeçalho.
`.cards { display: grid; grid-template-columns: repeat(3, 1fr); gap: 16px; }`	Coloca três cards lado a lado.
`.card, .painel { background: white; padding: 24px; border-radius: 16px; box-shadow: 0 8px 24px #18243a12; }`	Cria caixas arredondadas com sombra.
`.card h2 { font-size: 15px; color: #64748b; }`	Deixa os rótulos discretos.
`.card p { font-size: 26px; font-weight: bold; color: #3730a3; }`	Destaca valores numéricos.

CÓDIGO EXPLICADO

style.css • tabela e responsividade

CÓDIGO	EXPLICAÇÃO
`.painel { margin-top: 24px; overflow-x: auto; }`	Separa o painel e permite rolagem em telas estreitas.
`table { width: 100%; border-collapse: collapse; }`	A tabela usa toda a largura; remove espaços entre bordas.
`th, td { text-align: left; padding: 12px; border-bottom: 1px solid #e2e8f0; }`	Estiliza células e cabeçalhos.
`#status { color: #64748b; }`	Aplica cor à mensagem de status.
`@media (max-width: 680px) { .cards { grid-template-columns: 1fr; } }`	Em celular, empilha os cards em uma coluna.

CÓDIGO EXPLICADO

script.js • formatação de moeda

CÓDIGO

```
const moeda = valor => new
Intl.NumberFormat("pt-BR", {


  style: "currency", currency: "BRL"



}).format(valor);
```

EXPLICAÇÃO

Define função para transformar número em moeda brasileira.

Escolhe o formato monetário em reais.

Executa a formatação e retorna uma string como R$ 3.500,00.

CÓDIGO EXPLICADO

script.js • requisitando JSON

CÓDIGO

```
async function buscarJson(caminho) {



const resposta = await fetch(caminho);



if (!resposta.ok) throw new Error(`Erro HTTP
${resposta.status}`);


return resposta.json();



}
```

EXPLICAÇÃO

Cria uma função assíncrona que recebe o caminho de uma API.

Envia a requisição HTTP e espera a resposta.

Se a requisição falhar, cria um erro com o status.

Converte o corpo da resposta para objetos JavaScript.

Encerra a função.

CÓDIGO EXPLICADO

script.js • iniciando dashboard

CÓDIGO

```
async function carregarDashboard() {

const status = document.getElementById("status");

try {

const [indicadores, vendas] = await Promise.all([

buscarJson("/api/indicadores"),

buscarJson("/api/vendas")

]);
```

EXPLICAÇÃO

Define a função principal, assíncrona.

Localiza o parágrafo que mostra o andamento.

Inicia um bloco para capturar possíveis erros.

Busca as duas rotas ao mesmo tempo e aguarda ambas.

Pede os três indicadores numéricos.

Pede a lista de vendas.

Finaliza a chamada paralela.

CÓDIGO EXPLICADO

script.js • atualizando os indicadores

CÓDIGO	EXPLICAÇÃO
`document.getElementById("receita").textContent = moeda(indicadores.receitaTotal);`	Troca o texto do indicador Receita total pelo valor recebido.
`document.getElementById("total").textContent = indicadores.totalVendas;`	Mostra o número de vendas.
`document.getElementById("ticket").textContent = moeda(indicadores.ticketMedio);`	Mostra o ticket médio formatado em reais.

CÓDIGO EXPLICADO

script.js • construindo a tabela

CÓDIGO

```
const tabela = document.getElementById("tabela-vendas");


tabela.replaceChildren();


vendas.forEach(venda => {


const linha = document.createElement("tr");


[venda.produto, venda.categoria,
moeda(venda.valor)].forEach(valor => {
```

EXPLICAÇÃO

Seleciona o corpo da tabela HTML.

Apaga linhas antigas antes de preencher.

Percorre cada objeto da lista de vendas.

Cria uma nova linha da tabela.

Agrupa os três valores de cada venda e percorre-os.

CÓDIGO EXPLICADO

script.js • preenchendo a tabela

CÓDIGO

```
const celula = document.createElement("td");


celula.textContent = valor;


linha.appendChild(celula);


});


tabela.appendChild(linha);


});
```

EXPLICAÇÃO

Cria uma célula para cada informação.

Insere o texto de modo seguro, sem interpretar HTML.

Coloca a célula dentro da linha.

Termina o ciclo dos valores.

Adiciona a linha à tabela visível.

Termina o ciclo das vendas.

CÓDIGO EXPLICADO

script.js • sucesso e erros

CÓDIGO

```
status.textContent = `${vendas.length} vendas carregadas com
sucesso.`;


} catch (erro) {



status.textContent = `Não foi possível carregar os dados:
${erro.message}`;


console.error(erro);



}



}



carregarDashboard();
```

EXPLICAÇÃO

Informa ao usuário quantos registros apareceram.

Captura falhas de rede, API ou execução.

Mostra uma mensagem útil na página.

Exibe detalhes técnicos no console para depuração.

Fecha o tratamento de erro.

Fecha a função principal.

Executa o carregamento quando o script é iniciado.

ETAPA DA AULA

Como executar e testar

Rodar servidor, abrir navegador e acessar as rotas.

Terminal → dotnet run

Anotar a URL exibida: http://localhost:PORTA

Abrir /api/vendas e /api/indicadores

Abrir / e conferir os cards e a tabela

O que deve acontecer com os dados

CÓDIGO	EXPLICAÇÃO
`3500 + 100 + 600 + 900 + 1200`	Soma dos cinco valores registrados no backend.
`receitaTotal = 6300`	Receita calculada pela rota /api/indicadores.
`totalVendas = 5`	Quantidade de objetos na lista de vendas.
`ticketMedio = 6300 / 5 = 1260`	Média por registro de venda.
`Frontend → R$ 6.300,00 | 5 | R$ 1.260,00`	O JavaScript apresenta os valores formatados.

ETAPA DA AULA

Adaptação ao projeto dos alunos

Aproveite o HTML e o CSS já existentes, sem refazer a interface.

- Manter o design atual dos alunos
- Criar ou conferir ids: receita, total, ticket
- Reutilizar a chamada fetch para atualizar a tela
- Se quiser tabela, adicionar tbody id="tabela-vendas"

ETAPA DA AULA

Erros comuns e soluções

Use o console e confira o terminal.

404 → confira a rota ou a pasta wwwroot

fetch falhou → servidor parado ou porta incorreta

null em getElementById → ID ausente no HTML

CSS/JS não aparecem → verifique os caminhos dos arquivos

ETAPA DA AULA

Atividade prática

Faça alterações e observe o reflexo na interface.

- Adicionar duas vendas no array do Program.cs
- Alterar o preço do Notebook e recalcular receita
- Criar um quarto indicador opcional no backend
- Testar novamente as rotas e apresentar o resultado

ETAPA DA AULA

Fechamento da aula

Três conceitos que precisam ficar claros.

Frontend mostra a informação para o usuário

Backend processa e disponibiliza os dados

API conecta as duas partes via requisições HTTP

Próxima aula: receber cadastros pelo POST

REFERÊNCIA

Mapa do código completo

Program.cs — servidor, 5 vendas, GET /api/vendas e GET /api/indicadores

wwwroot/index.html — título, cards, status e tabela

wwwroot/css/style.css — layout, cards e responsividade

wwwroot/js/script.js — fetch, tratamento de erro e renderização

minidatasales.csproj — configuração do projeto .NET 8